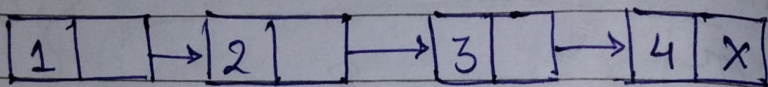


Question Bank

1] Explain Linked List and Its Types:
Ans. A linked list is a dynamic data structure in which elements form a sequential list. In linked list every node points to the next node in the list. Therefore, in a linked list, every node contains the following two types of data:

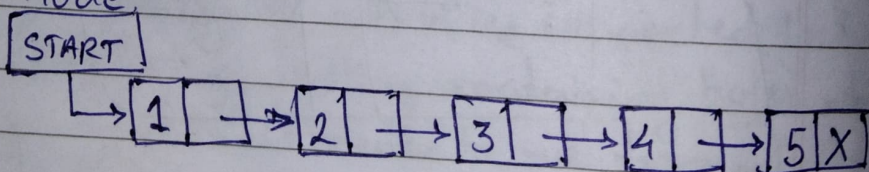
- The value of the node
 - A pointer or link to the next node
- The last node in the list contains a NULL pointer to indicate the end.



Types of linked list:

a) Singly Linked List:

A singly linked list is the simplest type of linked list in which every node contains some data and a pointer to the next node.

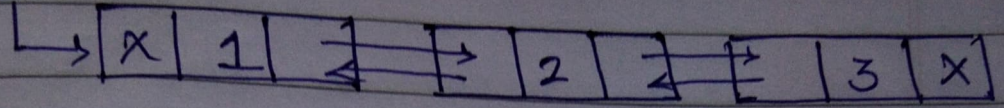


b) Doubly Linked List:

A doubly linked list is a more complex type of linked list which contains a pointer to the next as well as previous node in the

sequence. It consists of three parts - data, a pointer to the next node and a pointer to the previous node.

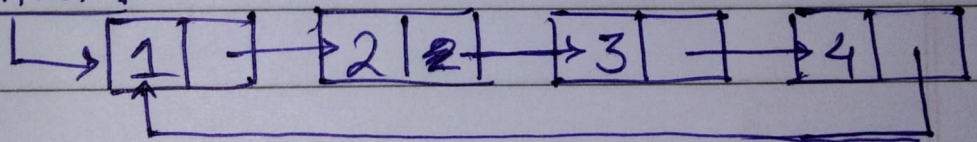
[START]



c) Circular Linked List:

In a circular linked list, the last node contains a pointer to the first node of the list. While traversing a circular linked list, we can begin at any node and traverse that list in any direction, forward or backward, until we reach the same node where we started. Thus, a circular linked list has no beginning and no ending.

[START]



2] ~~Q~~ Compare Array & Linked List

Array

Linked List

- | | |
|---|--|
| ① Arrays use contiguous memory locations, all elements are stored in adjacent memory blocks. This makes memory allocation static. | Linked lists use dynamic memory allocation, where each element is stored in a separate memory location and pointers are used to link them. |
| ② The size of an array is fixed at the time of declaration and cannot be changed during runtime. | The size of a linked list is dynamic, nodes can be added or removed during runtime. |
| ③ Array provide random access to elements, any element can be accessed directly using an index. | Linked lists provide sequential access. To access a particular element you must traverse the list from the head node to the desired node. |
| ④ Insertion and deletion require shifting elements making them inefficient, especially for middle or beginning operations. | Insertion and deletion are more efficient, as only pointer are needed, through finding the point of insertion/ |

7] Explain Different Tree Terminologies:

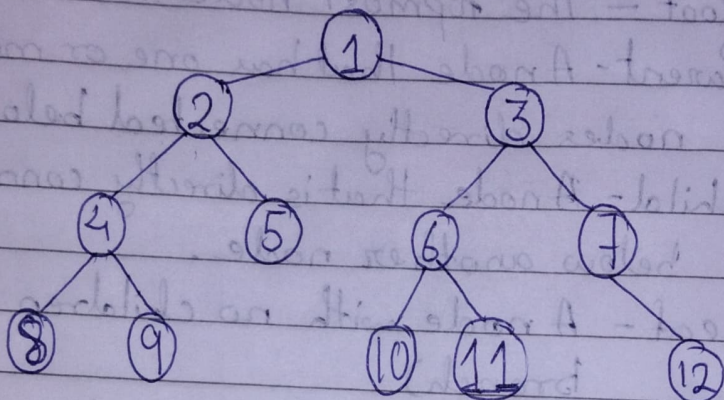
A Tree is a non-linear data structure which consists of a collection of nodes arranged in a hierarchical order.

Tree Terminologies:

- a. Node - A fundamental part of a tree that contains data. ~~and may have~~
- b. Root - The topmost node in the tree.
- c. Parent - A node that has one or more nodes directly connected below it.
- d. Child - A node that is directly connected below another node.
- e. Leaf - A node with no children (end of branch)
- f. Edge - A connection between two nodes.
- g. Level - The depth or distance of a node from the root
- h. Depth - Number of edges from the root to the node.
- i. Height - Number of edges on the longest path from a node down to leaf.
- j. Degree - Number of children a node has.
- k. Siblings - Nodes that share the same parent.
- l. Subtree - A tree consisting of a node and all its descendants.

8] Define Binary Tree and its Types?

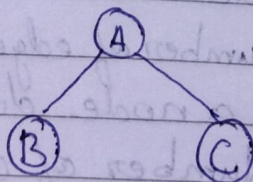
Ans. A Binary Tree is a hierarchical data structure where each node has at most two children: a left child and a right child. Binary trees are commonly used in applications like searching, sorting and representing structured data.



Types of Binary Tree

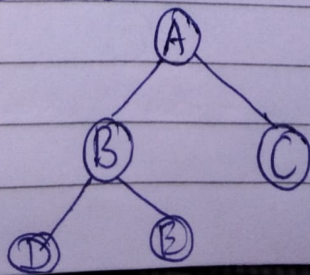
1. Full Binary Tree -

A tree in which every node has either 0 or 2 children.



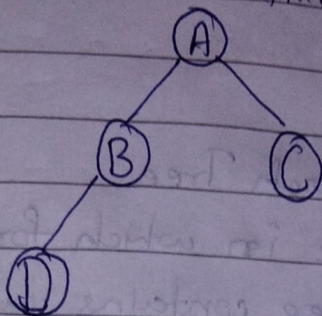
2. Perfect Binary Tree

A tree in which all internal nodes have 2 children and all leaf nodes are at the same level.



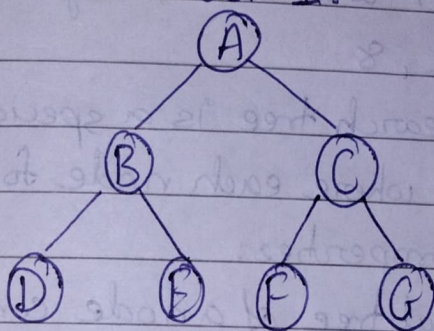
3. Complete Binary Tree:

All levels are completely filled except possibly the last, which is filled from left to right.



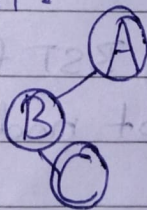
4. Balanced Binary Tree

A binary tree in which the height difference between the left and right subtree of any node is at most 1.



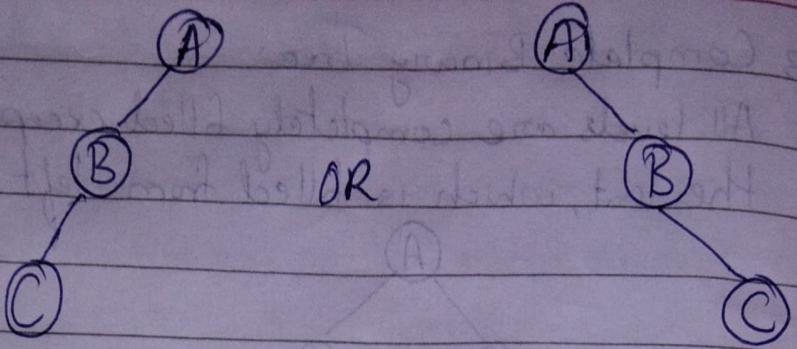
5. Degenerate (or Pathological) Binary Tree

A binary tree where each parent node has only one child, making it similar to linked list.



6. Skewed Binary Tree

A special type of degenerate tree where all nodes have only left child (left-skewed) or only right child (right-skewed).



7. Binary Search Tree

A binary tree in which for every node, the left subtree contains value less than node, and the right subtree contains values greater than the node.

Q] Define BST and Draw for given elements
7, 9, 6, 2, 8

Ans. A binary search tree is a special type of binary tree where each node follows the following properties:

- The left subtree of a node contains only nodes with values less than the node's value.
- The right subtree of a node contains only nodes with values greater than the node's value.

• Constructing BST for given elements

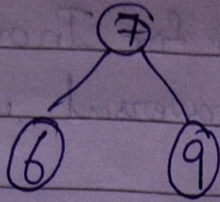
1. 7 → Root node

(7)

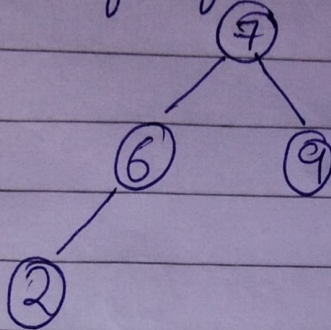
2. 9 → Greater than 7; goes to right

(7)
 (9)

3. 6 → Less than 7; goes to left

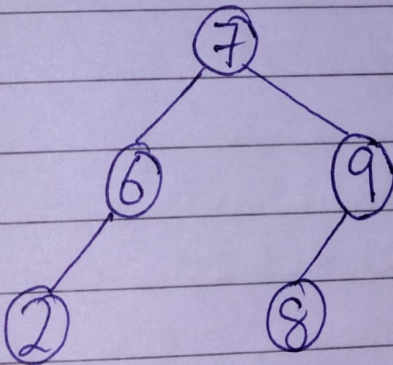


4. 2 $\rightarrow$ Less than 7, less than 6; goes to left of 6

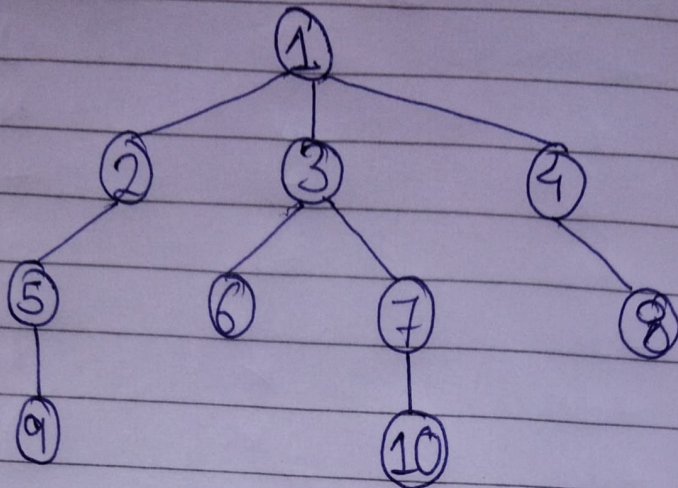


5. 8 $\rightarrow$ Greater than 7, less than 9; goes to left of 9

Final BST Structure:



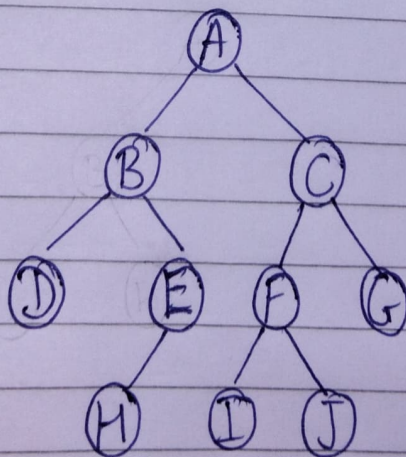
11] Give Inorder, Preorder and Post order Traversal for the following Binary Tree.



- Inorder $\rightarrow$ 9 5 2 6 10 7 3 8 4 1
- Preorder $\rightarrow$ 1 2 5 9 3 6 7 10 4 8
- Postorder $\rightarrow$ 9 5 2 6 10 7 3 8 4 1

12] From given traversals, construct the binary tree

Ans. Inorder: DBHEAIFJCG
Preorder: ABDEHCFIJG



13] Infix Expression to Postfix Expression

$$((A+B)*(C-D))/E*(d+n)$$

Character	Stack	Postfix Expression
(	(	empty
(	((	empty
A	((+)	A +
+	((+ +)	A d
B	((+ +)	AB (
)	(	AB+*
*	(*)*	AB+)
(	(* ()*	AB+)
C	(* (+)*	AB+C
-	(* (- +)*	AB+C
D	(* (- *	AB+CD
)	(*)	AB+CD-
)	empty	AB+CD*
/	/	AB+CD*÷
E	/	AB+CD*E
empty	empty	AB+CD*E/

Postfix Expression - AB+CD*E/

14] Infix to Prefix

 $(a+b)^*(c+d)$

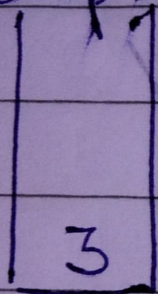
Character	Stack	Postfix Expression
)	)	empty
d	)	d
+	) +	d +
c	) +	d c +
(	empty	d c +
*	*	d c +
)	*)	d c +
b	*)	d c + b
+	*) +	d c + b
a	*) +	d c + b a
(	*	d c + b a +
empty	empty	d c + b a + *

Prefix Expression $\rightarrow * + a b + c d$ 15] Postfix Evaluation $3 \ 10 \ 5 \ + \ *$

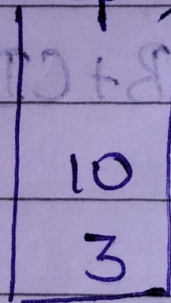
Step 1	Step 2	Step 3	Step 4	Step 5
3	10 3	5 10 3	15 3	45

16] Prefix Evaluation . $+ * 5 10 3$

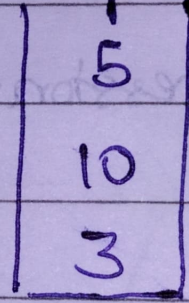
Step 1



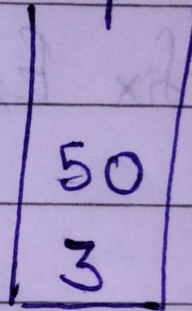
Step 2



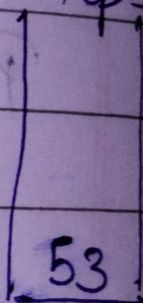
Step 3



Step 4



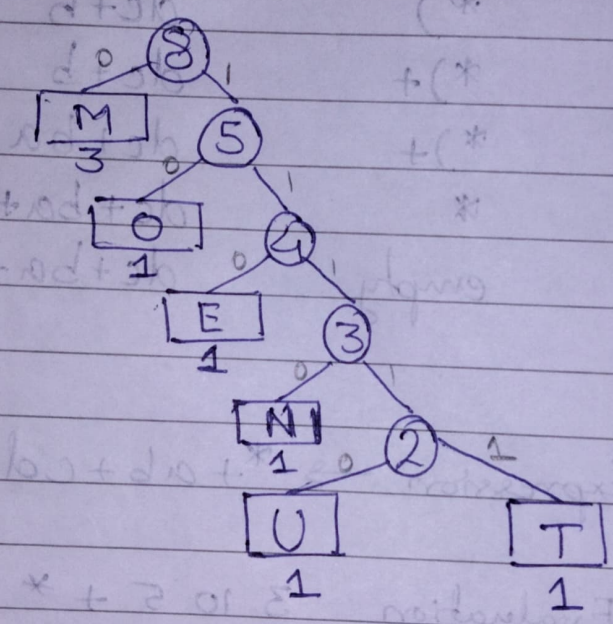
Step 5



17] Huffman Encoding tree and code

a) MOMENTUM

Char	Freq
M	3
O	1
E	1
N	1
T	1
U	1



M = 0

O = 10

E = 1100

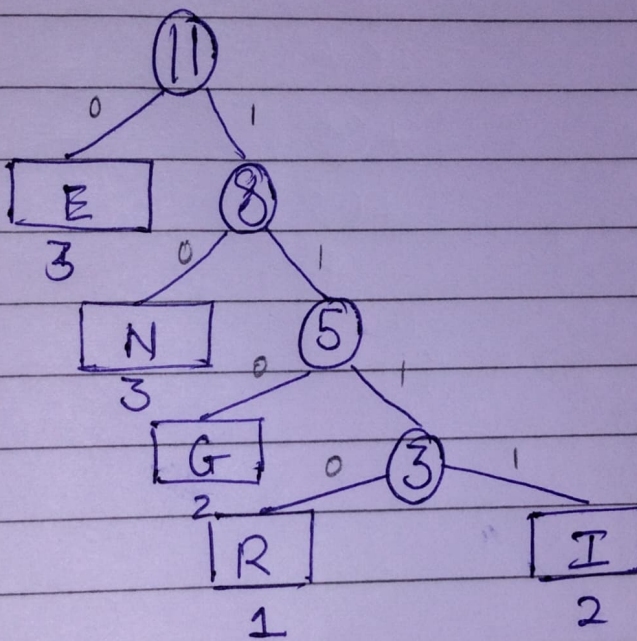
N = 1110

T = 11111

U = 11110

b) ENGINEERING

Char.	Freq.
E	3
N	3
G	2
I	2
R	1



E = 0

N = 10

G = 110

I = 1111

R = 1110